COS30018 — Intelligent Systems

Task C.4: Machine Learning 1

Dynamic Model Builder and Hyperparameter Experiments

Stock Prediction System v0.4

Student: Le Minh Kha
ID:104179506
Semester January 2026
Supervisor: Mr. Ky Trung

1. Introduction

This report documents the implementation of Task C.4 for the COS30018 Intelligent Systems unit. Task C.4 introduces a significant architectural improvement to the stock prediction system: a dynamic model builder function that constructs Deep Learning networks from configuration parameters rather than hardcoded layer definitions.

In previous versions (v0.1 through v0.3), the LSTM model was built by manually calling model.add() for each layer. This approach works for a single fixed architecture but makes experimentation cumbersome. Every time you want to test a different number of layers, a different cell type, or a different layer width, you need to edit the source code directly. Task C.4 solves this by introducing a create_model() function that accepts a list of layer configurations and dynamically constructs the corresponding Keras Sequential model.

The second component of this task is a systematic experiment grid that uses the new function to compare different Deep Learning architectures and training configurations. Four experiment groups were designed, each isolating a single variable: recurrent cell type, network depth, layer width, and training hyperparameters. The results quantify how each design choice affects prediction accuracy on CBA.AX stock data.

2. Requirements Analysis

Task C.4 specifies two main deliverables:

2.1 Requirement 1: Dynamic Model Builder Function

Write a function that takes as input several parameters including the number of layers, the size of each layer, and the layer name (e.g., LSTM, RNN, GRU, etc.) and returns a Deep Learning model. The task references the P1 project as an example of how this can be done, and requires the student to explain all code in detail.

2.2 Requirement 2: Architecture Experiments

Use the above function to experiment with different DL networks (LSTM, RNN, GRU, etc.) and with different hyperparameter configurations including different numbers of layers, layer sizes, number of epochs, and batch sizes.

3. Code Explanation: create_model()

This section explains the key components of the create_model() function, focusing on the non-trivial implementation details that required research and careful design.

3.1 Layer Type Mapping Dictionary

```
LAYER_MAP = {
    "LSTM": tf.keras.layers.LSTM,
    "GRU": tf.keras.layers.GRU,
    "SimpleRNN": tf.keras.layers.SimpleRNN,
    "Dense": tf.keras.layers.Dense
}

RECURRENT_TYPES = {"LSTM", "GRU", "SimpleRNN"}
```

The `LAYER_MAP` dictionary is the core mechanism that enables dynamic model construction. It maps human-readable string names to actual Keras layer classes. This is a common design pattern in Python called the **Strategy Pattern**: instead of using a chain of if/elif statements to instantiate different layer types, the dictionary provides direct class lookup in O(1) time. The `RECURRENT_TYPES` set identifies which layer types require special handling for `return_sequences` and dropout insertion.

Reference: TensorFlow Keras Layers API documentation (https://www.tensorflow.org/api_docs/python/tf/keras/layers)

3.2 The return_sequences Logic

```
def needs_return_sequences(index):
    """Check if any layer after index is a recurrent type."""
    for j in range(index + 1, len(layer_configs)):
        if layer_configs[j]["type"] in RECURRENT_TYPES:
            return True
    return False
```

This is the most critical piece of logic in the function. When stacking multiple recurrent layers, every recurrent layer **except the last one** must set `return_sequences=True`. Here is why: by default, a recurrent layer in Keras outputs only the final hidden state, producing a 2D tensor of shape `(batch_size, units)`. However, a subsequent recurrent layer expects a 3D input of shape `(batch_size, timesteps, features)`. Setting `return_sequences=True` tells the layer to output the hidden state at every timestep, preserving the 3D shape.

The function determines this automatically by scanning ahead through the remaining layer configurations. If any subsequent layer is a recurrent type, `return_sequences` is set to `True`. The last recurrent layer (which feeds into a Dense layer) returns only the final state, collapsing the sequence dimension.

Reference: Keras RNN documentation on return_sequences (https://keras.io/api/layers/recurrent_layers/)

3.3 Input Shape Assignment

```
if i == 0:
    model.add(layer_class(
        units,
```

```
        return_sequences=return_seq,
        input_shape=input_shape,
        **extra_params
    ))
```

Only the first layer in a Keras Sequential model requires the `input_shape` parameter. This tells the model the dimensions of the input data: `(n_steps, n_features)`, where `n_steps` is the number of historical days in each sequence (50 in our configuration) and `n_features` is the number of input columns (5: adjclose, volume, open, high, low). Subsequent layers infer their input shape automatically from the previous layer's output shape.

3.4 Dropout Insertion

```
if dropout_rate > 0:
    model.add(Dropout(dropout_rate))
```

A Dropout layer is added after each recurrent layer (but not after Dense layers). Dropout randomly sets a fraction of the input units to zero during training, which forces the network to learn more robust features that do not depend on specific neurons. The `dropout_rate=0.2` means 20% of neurons are randomly deactivated during each training step. Dropout is only applied after recurrent layers because the Dense output layer has a single neuron, and dropping it would mean losing the prediction entirely.

Reference: Srivastava et al. (2014), "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", JMLR 15.

3.5 Automatic Dense Output Layer

```
if layer_configs[-1]["type"] != "Dense":
    layer_configs = layer_configs + [{"type": "Dense", "units": 1}]
```

This safety check ensures the model always ends with a `Dense(1)` layer for single-value regression output. If the caller forgets to include it in their configuration, the function appends it automatically. Note the use of list concatenation (`+`) rather than `.append()` to avoid mutating the original list passed by the caller.

3.6 Function Signature and Validation

```
def create_model(input_shape, layer_configs, dropout_rate=0.2,
                 loss="mean_absolute_error", optimizer="adam"):
```

The function signature exposes the key hyperparameters as configurable arguments with sensible defaults. The `loss` defaults to Mean Absolute Error (MAE) because MAE is less sensitive to outliers than Mean Squared Error, which matters for financial data where occasional large price swings occur. The `optimizer` defaults to Adam (Adaptive Moment Estimation), which combines the advantages of RMSProp and momentum-based gradient descent. Input validation raises `ValueError` for empty configurations or unknown layer types, failing fast rather than producing cryptic TensorFlow errors downstream.

4. Experiment Design

To systematically evaluate the impact of different architectural and training choices, four experiment groups were designed. Each group isolates a single variable while holding all others constant. This controlled approach ensures that observed differences in performance can be attributed to the variable being tested rather than confounding factors.

All experiments use the same data configuration: CBA.AX stock data from 2020-01-01 to 2024-07-01, with 5 features (adjclose, volume, open, high, low), a 50-day lookback window, 1-day prediction horizon, and an 80/20 chronological train/test split. Random seeds are reset before each experiment (`np.random.seed(314)`) to ensure fair comparison.

4.1 Experiment 1: Cell Type Comparison

This experiment compares the three major recurrent cell architectures while keeping depth (2 layers), width (50 units), epochs (25), and batch size (32) constant.

Configuration	Cell Type	Layers	Units	Epochs	Batch Size
Exp1-LSTM-2x50	LSTM	2	50	25	32
Exp1-GRU-2x50	GRU	2	50	25	32
Exp1-SimpleRNN-2x50	SimpleRNN	2	50	25	32

LSTM (Long Short-Term Memory) uses three gates (forget, input, output) to control information flow, making it effective at learning long-range dependencies. GRU (Gated Recurrent Unit) simplifies this to two gates (reset, update), reducing parameters while maintaining similar performance. SimpleRNN has no gating mechanism, relying on a basic hidden-state update rule, which makes it susceptible to the vanishing gradient problem on longer sequences.

4.2 Experiment 2: Depth Comparison

This experiment tests how stacking more LSTM layers affects performance, with width (50 units), epochs (25), and batch size (32) held constant.

Configuration	Layers	Units per Layer	Total Parameters*
Exp2-Shallow-1L	1	50	~11,000
Exp2-Medium-2L	2	50	~31,000
Exp2-Deep-3L	3	50	~51,000

* Approximate trainable parameter counts excluding dropout layers and the final Dense layer. Deeper networks can learn more abstract feature hierarchies, but also introduce more parameters and longer training times. There is a risk of diminishing returns or even degraded performance if the network becomes too deep for the available data.

4.3 Experiment 3: Width Comparison

This experiment varies the number of units per layer while keeping depth (2 LSTM layers), epochs (25), and batch size (32) constant.

Configuration	Units per Layer	Epochs	Batch Size

Exp3-Narrow-25u	25	25	32
Exp3-Medium-50u	50	25	32
Exp3-Wide-100u	100	25	32

Wider layers increase the model's capacity to represent complex patterns. However, more units also means more parameters, increasing both training time and the risk of overfitting on limited data.

4.4 Experiment 4: Training Hyperparameters

This experiment varies epochs and batch size while keeping the architecture constant (2 LSTM layers, 50 units each).

Configuration	Epochs	Batch Size	Rationale
Exp4-A-e25b32	25	32	Baseline
Exp4-B-e50b32	50	32	Double epochs, same batch
Exp4-C-e25b16	25	16	Smaller batch (more updates per epoch)
Exp4-D-e50b64	50	64	More epochs + larger batch

More epochs give the optimizer more passes over the data to converge, but risk overfitting if the model begins memorising training patterns. Smaller batch sizes produce noisier gradient estimates, which can act as implicit regularisation, while larger batch sizes produce smoother but potentially less generalisable updates.

5. Execution Results

5.1 Model Summary

Figure 1 shows the model summary for the first experiment (Exp1-LSTM-2x50). The model consists of an LSTM layer with return_sequences=True (output shape: 50, 50), a Dropout layer, a second LSTM layer (output shape: 50), another Dropout layer, and a final Dense layer with a single output neuron. Total trainable parameters: 31,451.

```
============================================================
Running experiment 1/13: Exp1-LSTM-2x50
Group: 1-CellType
============================================================

Model Summary (first experiment):
Model: "sequential"
+-----------------------------------------------------------------------+
| Layer (type)                    | Output Shape           |       Param # |
|---------------------------------+------------------------+---------------|
| lstm (LSTM)                     | (None, 50, 50)         |        11,200 |
|---------------------------------+------------------------+---------------|
| dropout (Dropout)               | (None, 50, 50)         |             0 |
|---------------------------------+------------------------+---------------|
| lstm_1 (LSTM)                   | (None, 50)             |        20,200 |
|---------------------------------+------------------------+---------------|
| dropout_1 (Dropout)             | (None, 50)             |             0 |
|---------------------------------+------------------------+---------------|
| dense (Dense)                   | (None, 1)              |            51 |
+-----------------------------------------------------------------------+
 Total params: 31,451 (122.86 KB)
 Trainable params: 31,451 (122.86 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/25
```

Figure 1: Model summary for the baseline 2-layer LSTM architecture (Exp1-LSTM-2x50)

5.2 Training Output

Figure 2 shows the console output during training of the first experiment, displaying per-epoch loss and validation loss metrics. The loss steadily decreases across epochs, indicating the model is learning.

```
452
453  [1m1/7[0m [32m==[0m[37m====================[0m [1m2s[0m 441ms/step
454  [1m6/7[0m [32m==================[0m[37m===[0m [1m0s[0m 11ms/step
455  [1m7/7[0m [32m====================[0m[37m[0m [1m0s[0m 81ms/step
456  [1m7/7[0m [32m====================[0m[37m[0m [1m1s[0m 85ms/step
457  C:\Users\admin\AppData\Local\Programs\Python\Python311\Lib\site-packages\keras\src\layers\rnn\rnn.py:200: Us
458    super().__init__(**kwargs)
459
460  --- Results for Exp1-LSTM-2x50 ---
461    MAE:  $3.54
462    RMSE: $4.03
463    MAPE: 3.33%
464    R◆:   0.7921
465    Training time: 31.1s
466
467  ============================================================
468  Running experiment 2/13: Exp1-GRU-2x50
469  Group: 1-CellType
470  ============================================================
471  Epoch 1/25
472
473  [1m 1/25[0m [37m====================[0m [1m2:22[0m 6s/step - loss: 0.3311 - mean_absolute_error:
474  [1m 3/25[0m [32m==[0m[37m==================[0m [1m0s[0m 40ms/step - loss: 0.2826 - mean_absolu
475  [1m 5/25[0m [32m====[0m[37m================[0m [1m0s[0m 38ms/step - loss: 0.2481 - mean_absolu
476  [1m 7/25[0m [32m=====[0m[37m===============[0m [1m0s[0m 38ms/step - loss: 0.2286 - mean_absolu
```

Figure 2: Training output showing epoch-by-epoch loss progression

5.3 Experiment Results Summary

Figure 3 shows the complete results summary as printed by the experiment runner. Table 1 below presents the same data in a formatted table.

```
p - loss: 0.0322 - mean_absolute_error: 0.0322 - val_loss: 0.0203 - val_mean_absolute_error: 0.0203
7/7 ━━━━━━━━━━━━━━━━━━━━ 1s 82ms/step

--- Results for Exp4-D-e50b64 ---
  MAE:  $3.59
  RMSE: $4.15
  MAPE: 3.36%
  R²:   0.7785
  Training time: 37.3s

================================================================================
EXPERIMENT RESULTS SUMMARY
================================================================================
         Experiment         Group  MAE ($)  RMSE ($)  MAPE (%)     R²  Time (s)
     Exp1-LSTM-2x50    1-CellType     3.54      4.03      3.33 0.7921      29.8
      Exp1-GRU-2x50    1-CellType     1.50      1.82      1.43 0.9577      32.4
Exp1-SimpleRNN-2x50    1-CellType     6.68      6.98      6.35 0.3739      16.7
    Exp2-Shallow-1L       2-Depth     1.10      1.38      1.06 0.9756      16.4
     Exp2-Medium-2L       2-Depth     3.54      4.03      3.33 0.7921      28.1
       Exp2-Deep-3L       2-Depth     2.18      2.76      2.04 0.9022      39.0
    Exp3-Narrow-25u       3-Width     3.82      4.49      3.56 0.7408      25.9
    Exp3-Medium-50u       3-Width     3.54      4.03      3.33 0.7921      28.3
     Exp3-Wide-100u       3-Width     2.26      2.74      2.12 0.9033      35.9
      Exp4-A-e25b32 4-HyperParams     3.54      4.03      3.33 0.7921      28.6
      Exp4-B-e50b32 4-HyperParams     1.48      1.89      1.39 0.9541      51.2
      Exp4-C-e25b16 4-HyperParams     1.78      2.23      1.66 0.9359      40.8
      Exp4-D-e50b64 4-HyperParams     3.59      4.15      3.36 0.7785      37.3

--- 1-CellType ---
         Experiment      Group  MAE ($)  RMSE ($)  MAPE (%)     R²  Time (s)
     Exp1-LSTM-2x50 1-CellType     3.54      4.03      3.33 0.7921      29.8
      Exp1-GRU-2x50 1-CellType     1.50      1.82      1.43 0.9577      32.4
Exp1-SimpleRNN-2x50 1-CellType     6.68      6.98      6.35 0.3739      16.7

--- 2-Depth ---
     Experiment   Group  MAE ($)  RMSE ($)  MAPE (%)     R²  Time (s)
Exp2-Shallow-1L 2-Depth     1.10      1.38      1.06 0.9756      16.4
 Exp2-Medium-2L 2-Depth     3.54      4.03      3.33 0.7921      28.1
 Exp3-Wide-100u 3-Width     2.26      2.74      2.12 0.9033      35.9

--- 4-HyperParams ---
   Experiment         Group  MAE ($)  RMSE ($)  MAPE (%)     R²  Time (s)
Exp4-A-e25b32 4-HyperParams     3.54      4.03      3.33 0.7921      28.6
Exp4-B-e50b32 4-HyperParams     1.48      1.89      1.39 0.9541      51.2
Exp4-C-e25b16 4-HyperParams     1.78      2.23      1.66 0.9359      40.8
Exp4-D-e50b64 4-HyperParams     3.59      4.15      3.36 0.7785      37.3

Results saved to 'experiment_results_v04.csv'
```

Figure 3: Console output of the complete experiment results summary

Table 1: Complete Experiment Results

Experiment	Group	MAE ($)	RMSE ($)	MAPE (%)	R^2	Time (s)
Exp1-LSTM-2x50	1-CellType	3.54	4.03	3.33	0.7921	29.8
Exp1-GRU-2x50	1-CellType	1.50	1.82	1.43	0.9577	32.4
Exp1-SimpleRNN-2x50	1-CellType	6.68	6.98	6.35	0.3739	16.7
Exp2-Shallow-1L	2-Depth	1.10	1.38	1.06	0.9756	16.4
Exp2-Medium-2L	2-Depth	3.54	4.03	3.33	0.7921	28.1
Exp2-Deep-3L	2-Depth	2.18	2.76	2.04	0.9022	39.0
Exp3-Narrow-25u	3-Width	3.82	4.49	3.56	0.7408	25.9
Exp3-Medium-50u	3-Width	3.54	4.03	3.33	0.7921	28.3
Exp3-Wide-100u	3-Width	2.26	2.74	2.12	0.9033	35.9
Exp4-A-e25b32	4-HyperParams	3.54	4.03	3.33	0.7921	28.6
Exp4-B-e50b32	4-HyperParams	1.48	1.89	1.39	0.9541	51.2
Exp4-C-e25b16	4-HyperParams	1.78	2.23	1.66	0.9359	40.8
Exp4-D-e50b64	4-HyperParams	3.59	4.15	3.36	0.7785	37.3

6. Results Analysis

6.1 Cell Type Comparison (Experiment 1)

Cell Type	MAE ($)	RMSE ($)	MAPE (%)	R^2	Time (s)
LSTM	3.54	4.03	3.33	0.7921	29.8
GRU	1.50	1.82	1.43	0.9577	32.4
SimpleRNN	6.68	6.98	6.35	0.3739	16.7

GRU delivered the strongest results across every metric, achieving an R^2 of 0.9577 and MAPE of 1.43%, outperforming LSTM (R^2 0.7921, MAPE 3.33%) by a substantial margin. This is a noteworthy result. GRU uses fewer parameters than LSTM (two gates instead of three), which can reduce overfitting on smaller datasets. With our relatively modest dataset (~1,100 trading days), GRU's simpler parameterisation may be an advantage rather than a limitation.

SimpleRNN performed poorly (R^2 0.3739, MAPE 6.35%), which is expected. Without gating mechanisms, SimpleRNN struggles with sequences of length 50 due to the vanishing gradient problem. The hidden state rapidly loses information from earlier timesteps, making it unable to capture the medium-to-long-term dependencies that matter for stock prediction.

Training times were comparable for LSTM and GRU (~30s), while SimpleRNN was faster (16.7s) due to its simpler computation per timestep.

6.2 Depth Comparison (Experiment 2)

Depth	MAE ($)	RMSE ($)	MAPE (%)	R^2	Time (s)
1 Layer (Shallow)	1.10	1.38	1.06	0.9756	16.4
2 Layers (Medium)	3.54	4.03	3.33	0.7921	28.1
3 Layers (Deep)	2.18	2.76	2.04	0.9022	39.0

The single-layer LSTM produced the best results in this group, with an R^2 of 0.9756 and MAPE of just 1.06%. This is the best performance across **all 13 experiments**. Adding more layers did not improve performance; the 2-layer model dropped to R^2 0.7921, and while the 3-layer model recovered somewhat to R^2 0.9022, it did not match the single-layer result.

This counter-intuitive result has a practical explanation. For our data size and prediction task (single-day-ahead regression on ~1,100 samples), a single LSTM layer already has sufficient capacity to model the relevant patterns. Additional layers introduce more parameters that need to be trained, and with limited data, the optimizer may not converge to an optimal solution within 25 epochs. The additional dropout layers between stacked LSTMs also remove information that may have been useful.

Training time scaled roughly linearly with depth: 16.4s for 1 layer, 28.1s for 2 layers, 39.0s for 3 layers.

6.3 Width Comparison (Experiment 3)

Width	MAE ($)	RMSE ($)	MAPE (%)	R^2	Time (s)

25 units (Narrow)	3.82	4.49	3.56	0.7408	25.9
50 units (Medium)	3.54	4.03	3.33	0.7921	28.3
100 units (Wide)	2.26	2.74	2.12	0.9033	35.9

Performance improved monotonically with width. The wide configuration (100 units) achieved R^2 0.9033 and MAPE 2.12%, a significant improvement over the narrow (R^2 0.7408) and medium (R^2 0.7921) configurations. This suggests the 2-layer architecture with only 25 or 50 units was capacity-limited and could not represent the full complexity of the input patterns.

The trade-off is training time, which increased from 25.9s to 35.9s. However, this 39% increase in time bought a 22% reduction in R^2 error ($1 - R^2$ dropped from 0.26 to 0.10), which is a favourable exchange.

6.4 Hyperparameter Comparison (Experiment 4)

Config	Epochs	Batch	MAE ($)	RMSE ($)	MAPE (%)	R^2	Time (s)
A (baseline)	25	32	3.54	4.03	3.33	0.7921	28.6
B (more epochs)	50	32	1.48	1.89	1.39	0.9541	51.2
C (small batch)	25	16	1.78	2.23	1.66	0.9359	40.8
D (more + large)	50	64	3.59	4.15	3.36	0.7785	37.3

Doubling epochs from 25 to 50 with batch size 32 (Config B) dramatically improved performance from R^2 0.7921 to 0.9541, suggesting the baseline 25 epochs was **insufficient for full convergence**. The model was still actively learning when training stopped.

Reducing batch size from 32 to 16 (Config C) also improved results substantially (R^2 0.9359 vs 0.7921), even with the same 25 epochs. Smaller batches mean more gradient updates per epoch (the optimizer takes more steps, albeit noisier ones), which can accelerate convergence.

The most interesting result is Config D (50 epochs, batch 64), which performed almost identically to the 25-epoch baseline (R^2 0.7785 vs 0.7921). Despite having double the epochs, the large batch size meant fewer gradient updates per epoch, and the combined effect essentially cancelled out the benefit of additional training.

7. Key Findings

Finding 1: GRU outperformed LSTM on this dataset. This challenges the common assumption that LSTM is always the superior choice. For smaller datasets with moderate sequence lengths, GRU's reduced parameter count can be advantageous. The three-gate LSTM architecture may be over-parameterised for this prediction task.

Finding 2: Deeper is not necessarily better. The single-layer LSTM achieved the best overall performance (R^2 0.9756, MAPE 1.06%). For single-step-ahead regression on limited financial data, one well-configured layer outperforms stacked architectures. This finding aligns with the principle that model complexity should match data complexity.

Finding 3: Width matters more than depth for this task. Increasing layer width from 25 to 100 units consistently improved performance within the 2-layer architecture, while increasing depth from 1 to 3 layers did not. Width increases the representational capacity of each layer without introducing the information bottlenecks that stacking creates.

Finding 4: Training duration had the largest impact on 2-layer LSTM performance. Doubling epochs from 25 to 50 improved R^2 from 0.7921 to 0.9541, the single largest improvement within any experiment group (for the 2-layer 50-unit architecture). The baseline 25 epochs was clearly insufficient for convergence.

Finding 5: Batch size interacts with epoch count in non-obvious ways. Larger batch sizes require more epochs to achieve the same number of gradient updates. Config D (50 epochs, batch 64) performed worse than Config B (50 epochs, batch 32) despite identical epoch counts, because the larger batch size meant roughly half the total gradient updates.

8. Challenges and Solutions

Challenge 1: Automatic return_sequences detection. The most complex part of create_model() was determining when to set return_sequences=True. A recurrent layer must output sequences if any subsequent layer is also recurrent, but the function cannot know this without scanning ahead in the configuration list. The solution was the needs_return_sequences() helper function that performs a forward lookahead through the remaining layer configs.

Challenge 2: Seed resetting between experiments. Initial runs produced inconsistent results because the random state accumulated across experiments. Resetting all three random seeds (numpy, TensorFlow, Python random) before each experiment ensures that weight initialisation, data shuffling, and dropout patterns are identical, making results comparable.

Challenge 3: Total training time. Running 13 experiments sequentially required approximately 7 minutes of total computation. This was managed by printing per-experiment progress indicators and accumulating results in a list for batch display at the end, rather than requiring manual recording.